

ふんわり掴むAttention ~vol2~

Dot_product_attention, efficient_attention, SE
Transformer, Linformer(encoderのみ)

前回のおさらい

Attentionは「大域的情報」！！

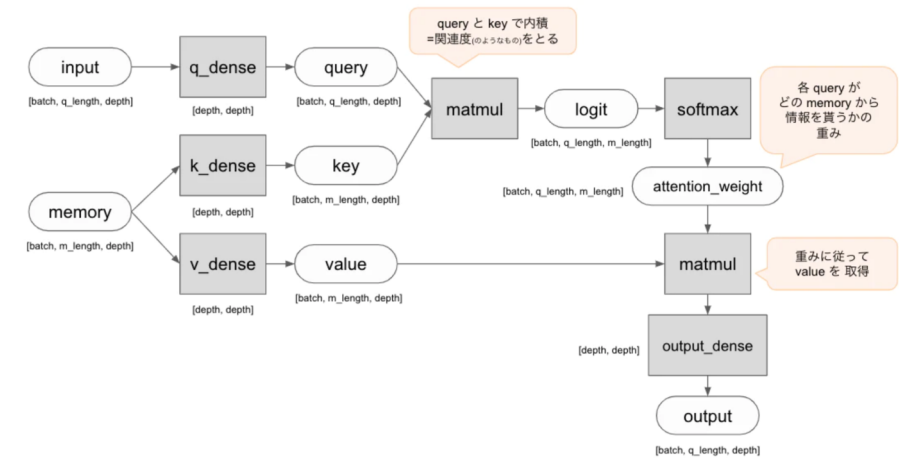
... 入力に対して俯瞰する

- Dot_product_attentionは全結合と行列積くらいで構成

$$\cdots D(Q,K,V) = \rho(QK^T)V$$

計算量が系列長を n としたときに $O(n^2)!!$

$$E(Q,K,V) = \rho_q(Q)(\rho_k(K)^TV)$$
 という工夫がある

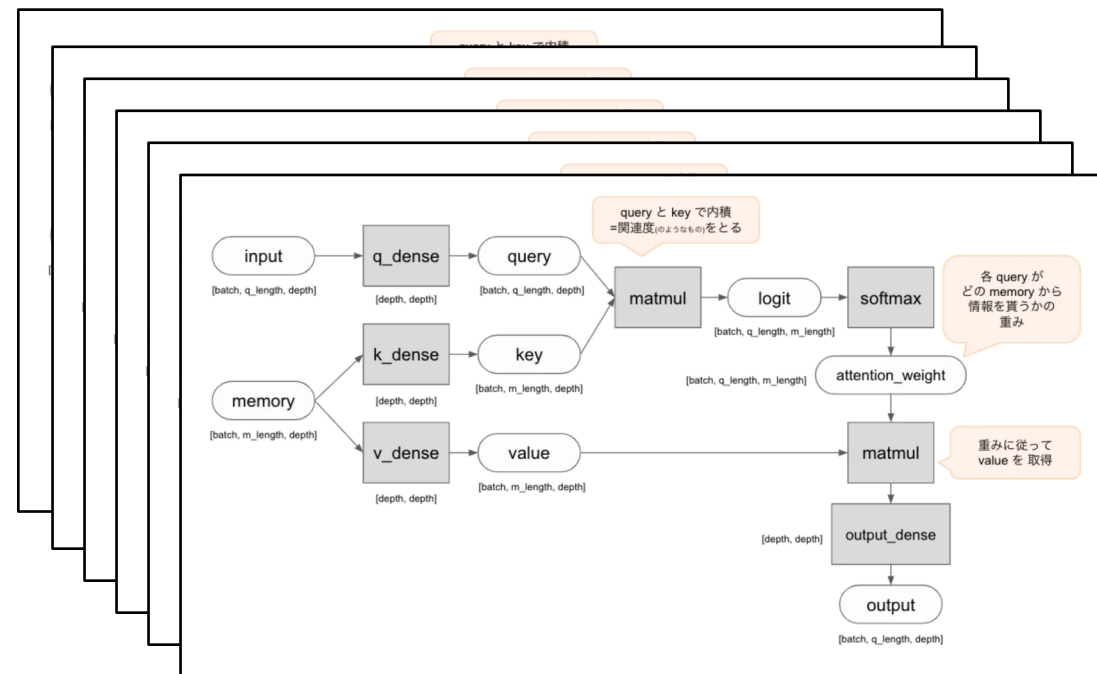


Dot_product_attention(multi_head)

[Input] → [input₁, input₂ ..., input_n] →
(batch, seq_len, depth)
→ (batch, seq_len, n, depth/n)

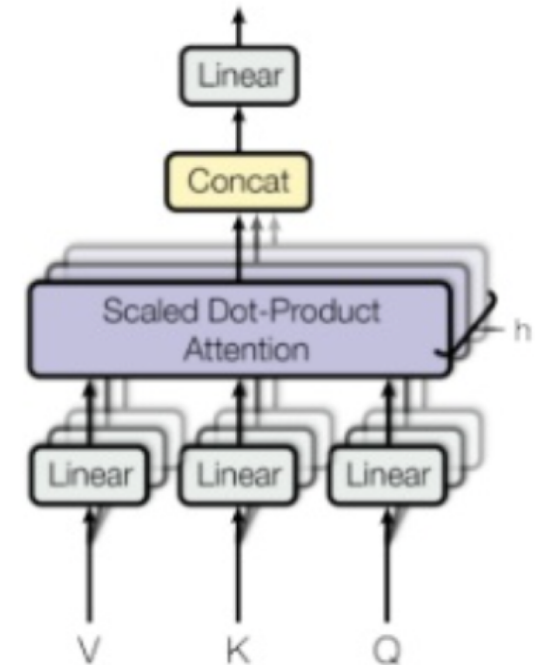
[output₁, output₂ ..., output_n] ←

各々は、(batch, seq_len, n, depth/n) ... 結合して
(batch, seq_len, depth) これを全結合層に通して (batch, seq_len, depth)
に



Dot_product_attention(multi_head)

- multi_headの良い点
- QKVを作るための全結合層がn種類あり、多様な表現を獲得できる
- 複数の部分に注目できるようになる
…系列の一部に引っ張られすぎがなくなる



`torch.nn.MultiheadAttention`

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

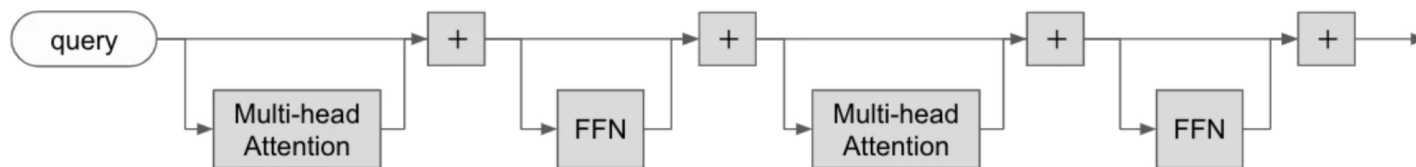
where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

FeedForwardNetwork

Attentionをした後は、全結合層のブロックに通す

これは`nn.Linear(hidden_dim, hidden_dim*4)`,
`nn.ReLU`,`nn.Dropout()`,`nn.Linear(hidden_dim*4, hidden_dim)`で
構成

下のようにブロックを積み重ねる



Mask…見たくない部分を隠す

- QK^T にmaskをかけて、見たくない部分のattention_weightは0にする
 - 見たくないもの1：padding
 - 見たくないもの2：decoderでの未来の情報(今回は触れません)
 - 具体的にはdatasetで系列長に沿ったmaskを用意しておく
(seq_len, seq_len)
 - ① `mask = np.array([1] * len_inp + [0] * (self.max_len - len_inp))`
 - `mask = np.einsum("n,m -> nm", mask, mask)`
- `attention_weight = attention_weight * mask if mask else: attention_weight`

```
[
    おはよう / <PAD> / <PAD>,
    インコ   / が      / 好き,
    お腹     / へった / <PAD>,
]
```

Positional Encoding

- 系列の順序をしっかりと扱うためのもの
- BERTでは変数として学習
- 詳しくは割愛(間に合わず…)
- これがじゃまだった という解法もある(DSB2019 3rd)

Transformerの今後

- 計算量を減らそう！！系の取り組みは数多い
- その中でLinformer を紹介
- 時間計算量は $O(n^2) \rightarrow O(nk)$, $k = \text{dim} * \log(\text{dim})$ 程度
- <https://arxiv.org/abs/2006.04768>

Linformer: Self-Attention with Linear Complexity

- $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)$ *WO*

$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

では、 q, k, v は $\text{dim}/\text{num_heads}$ 次元であり、実際に調べてみると
QKの**ランクが意外と低い**ことがわかった！

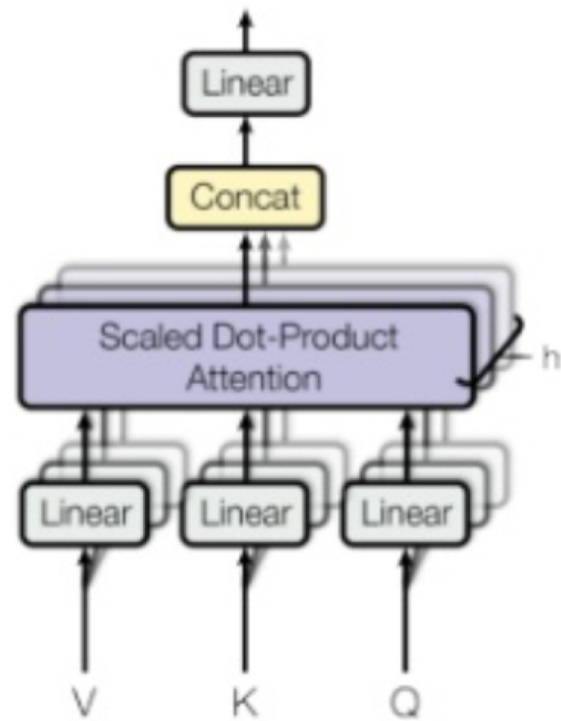
…QKの情報は高次元ではない(**低次元で表現しても良くない！？**)

+ 層を経るごとにこの傾向は強まっていく

QKの低次元への射影を試みる

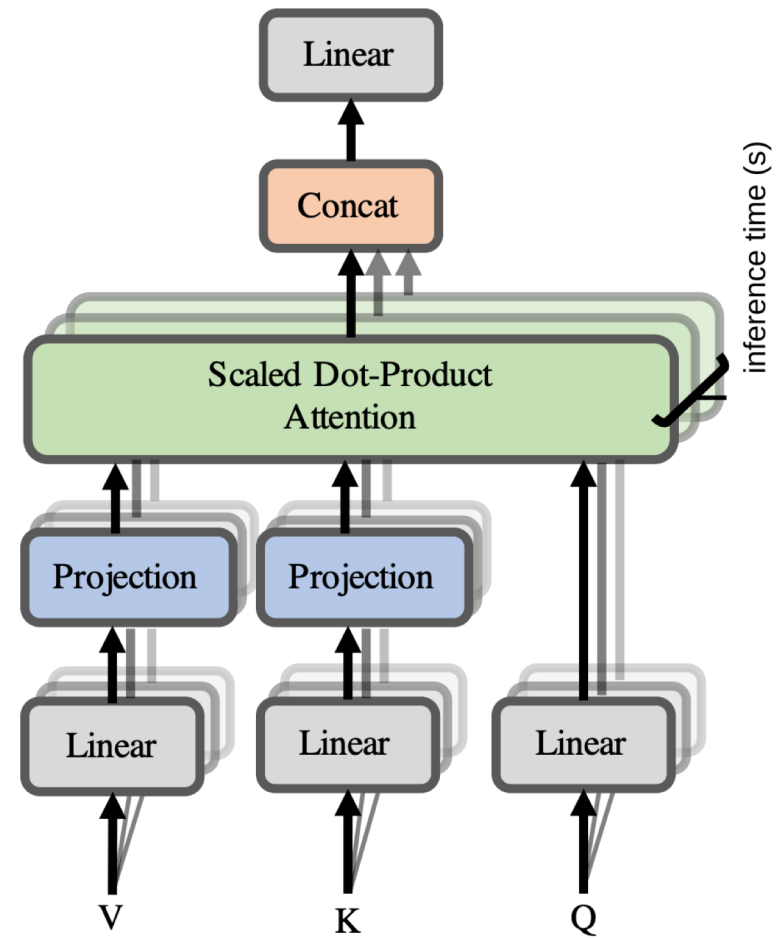
- $D(Q,K,V) = \rho(QK)V$ について、QKのランクが低くても、softmaxをかけた後の $\rho(QK)$ のランクも低くないとまずい？
…ある定理により、 $\rho(QK) \doteq \rho(QK) R^T R$ でOK!
(R は $k \times n$ 次元で $N(0, 1/k)$ 、 $k = \log(n) \times \text{定数}$)
- このままだと $O(n^2) \rightarrow O(n \log(n))$ でそんなに早くない？
… $k = d \log(d) \times \text{定数}$ ぐらいでOKで、
 $L(Q,K,V) = \rho(Q(EK))(FV)$.. E, F は学習可能な行列(固定でも可)

外観の比較



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

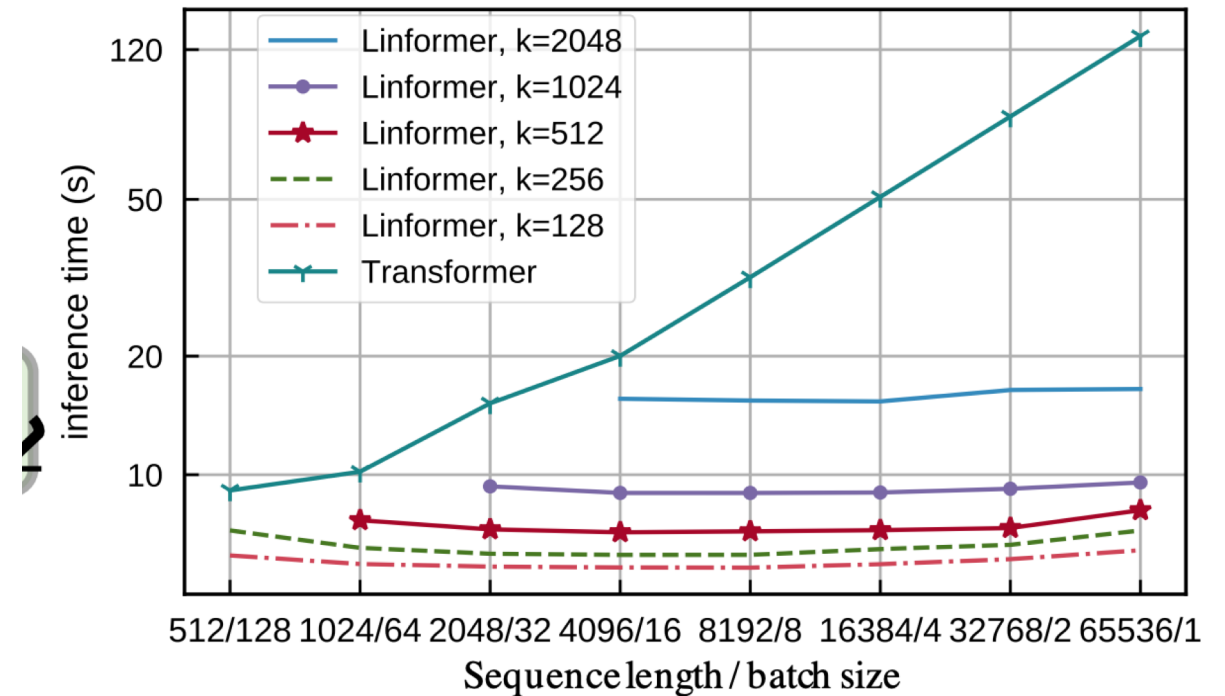
where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



性能大丈夫？

Transformerは系列長に比例して
推論時間増加

Linformerの推論時間はkによる
確かに速い



性能大丈夫？

- Transformerと遜色ない
- 性能は k 依存(n/k ではない)
- …同じ k なら n が長くても性能一緒

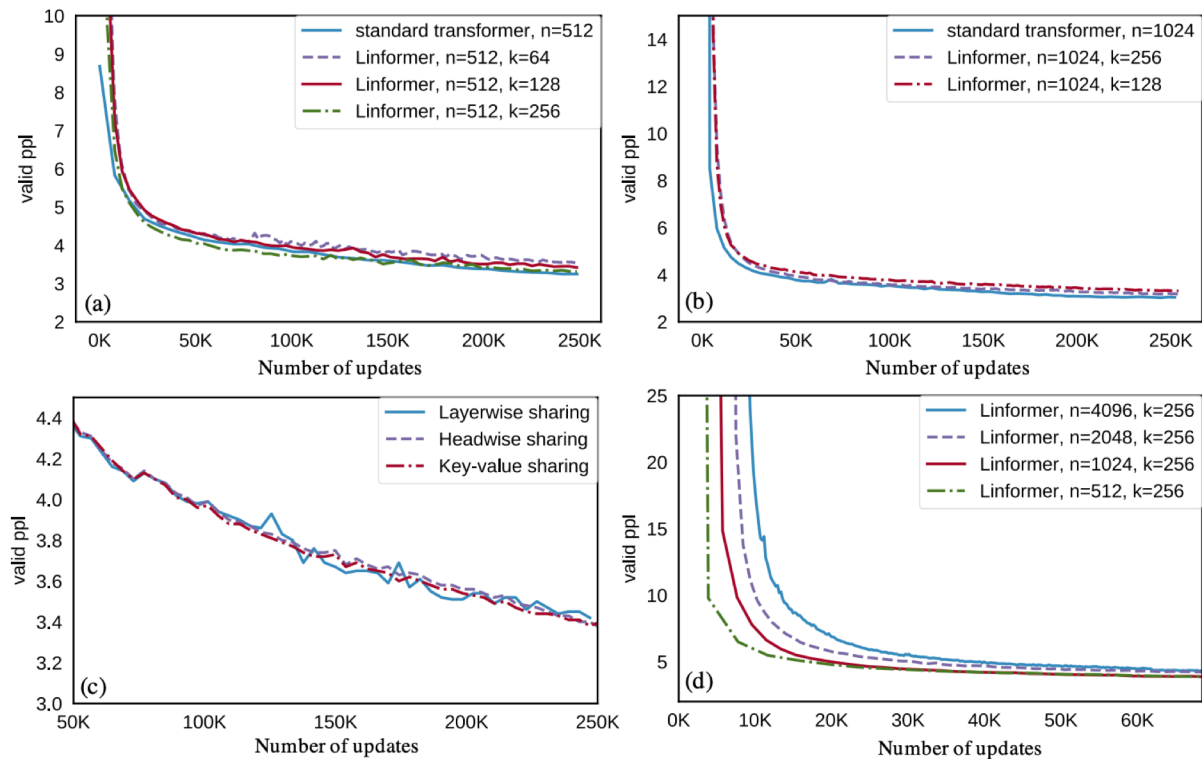


Figure 3: Pretraining validation perplexity versus number of updates.

補足

- NLPではBatch NormよりもLayer Normを採用
- Linformerはfacebookがユーザーからの報告よりはやくヘイトスピーチ検出をするために開発したらしい
- LinformerではQKにmaskをするのではなく、投射するKVにmaskをする
- QKではなく $QK/\text{depth}^{0.5}$ (Qは $(\text{bs}, \text{seq_len}, \text{depth})$ 次元) というスケールリングをする

参考

- https://www.youtube.com/watch?v=-_2AF9Lhweo
- <https://github.com/tatp22/linformer-pytorch>
- <https://qiita.com/halhorn/items/c91497522be27bde17ce>
- <https://www.youtube.com/watch?v=qjO7Mdl8x-8>

次回

- 簡単な埋め込み表現の獲得方法2つ(`nn.Embedding`, `w2v`)
- 実際に使われたtransfomerモデル(コンペで)
- <https://blog.brainpad.co.jp/entry/2021/02/17/140000>